

Greedy Algorithms



Greedy Algorithms

Repeatedly choose a best choice **NOW**.
 (“Locally optimal” choices, no “undo”s.)
 Prove that you end up with a global optimum.



Proof Technique: Induction + “greedy choice” / “exchange” argument

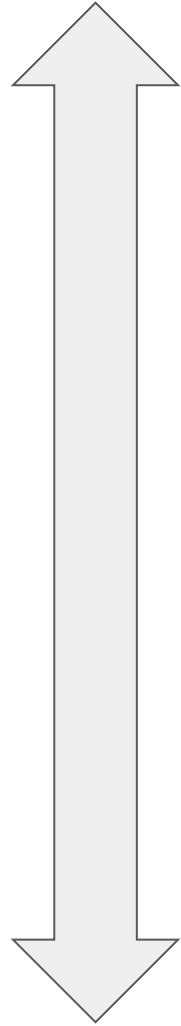
Warning: Greed is generally bad!



Greedy

Divide and conquer

Dynamic programming



- Fast
- Doesn't work for most problems
- Often slower than greedy and faster than DP
- Works when solutions to disjoint subproblems can be combined into final solution
- Generally slower (but still usually efficient)
- Applies to many problems

⋮

harder problems where none of these methods apply (coming soon)

But sometimes greed can be good...

Unweighted Course Registration

(aka Task Scheduling)

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

Foundations of CS

Blah

Applied Topology with Doughnuts

Advanced Blah

Goal: Choose a **largest*** number of non-intersecting courses
(there may be many optimal solutions—we seek any one!)

*We do not endorse this course registration strategy

But sometimes greed can be good...

Unweighted Course Registration

(aka Task Scheduling)

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

Foundations of CS

Blah

Applied Topology with Doughnuts

Advanced Blah

Goal: Choose a **largest*** number of non-intersecting courses
(there may be many optimal solutions—we seek any one!)

*We do not endorse this course registration strategy

But sometimes greed can be good...

Unweighted Course Registration

(aka Task Scheduling)

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

Foundations of CS

Blah

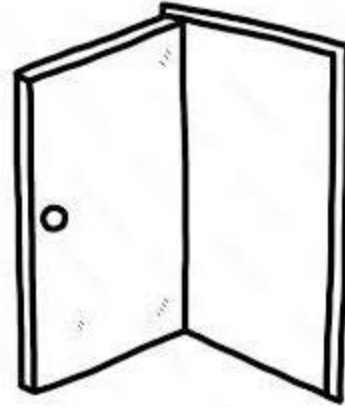
Applied Topology with Doughnuts

Advanced Blah

Goal: Choose a **largest*** number of non-intersecting courses
(there may be many optimal solutions—we seek any one!)

*We do not endorse this course registration strategy

I have not 1, not 2, but **3**
greedy algorithms
that solve this problem!



Professor Y's Greedy Algorithms



Attempt 1: Choose a **shortest interval** (breaking ties arbitrarily), take it, remove overlaps, iterate on remaining intervals.

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

Foundations of CS

Blah

Applied Topology with Doughnuts

Advanced Blah

Counterexample:

What makes this a greedy algorithm?



Professor Y's Greedy Algorithms



Attempt 1: Choose a **shortest interval** (breaking ties arbitrarily), take it, remove overlaps, iterate on remaining intervals.

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

Foundations of CS

Blah

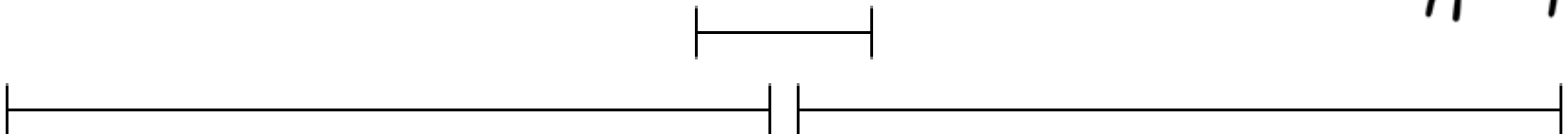
Applied Topology with Doughnuts

Advanced Blah

What makes this a greedy algorithm?



Counterexample:



Professor Y's Greedy Algorithms



Attempt 2: Choose an **interval that starts earliest**, take it, remove overlaps, iterate on remaining intervals.

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

Foundations of CS

Blah

Applied Topology with Doughnuts

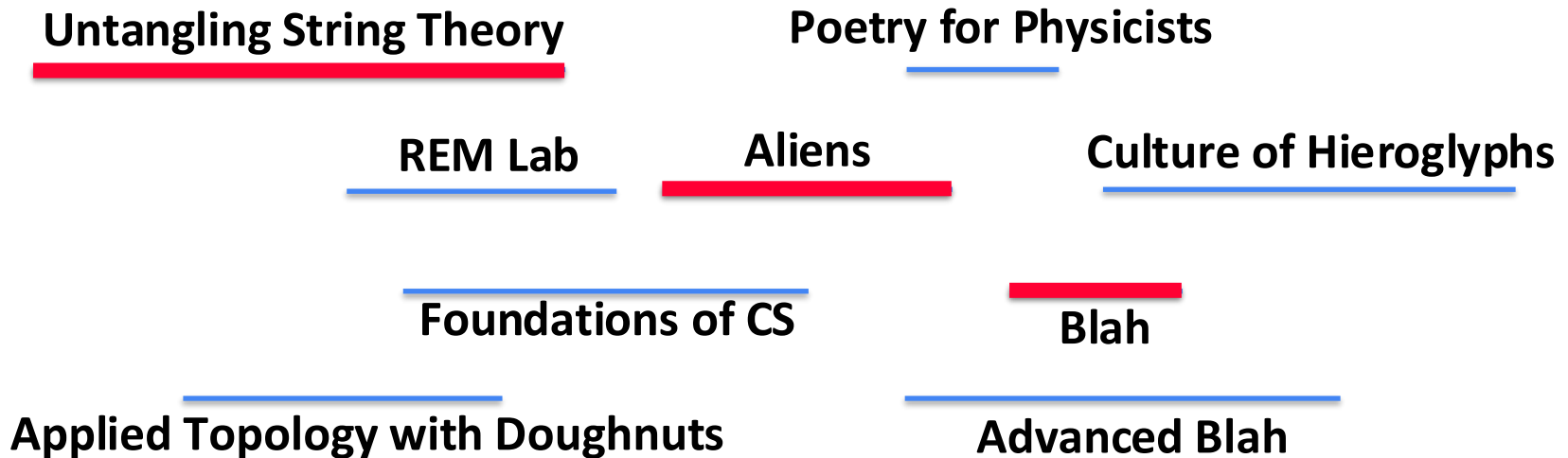
Advanced Blah

Counterexample:

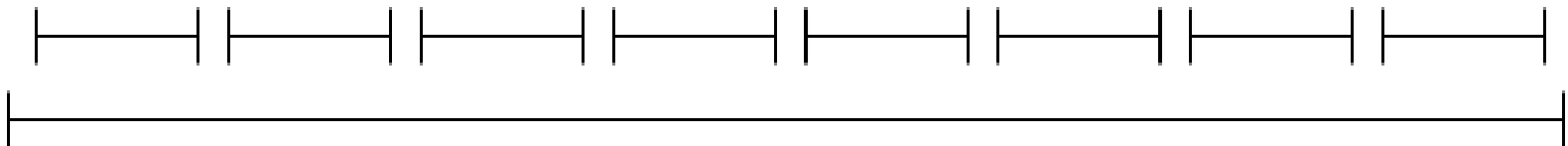
Professor Y's Greedy Algorithms



Attempt 2: Choose an **interval that starts earliest**, take it, remove overlaps, iterate on remaining intervals.



Counterexample:



Professor Y's Greedy Algorithms



Attempt 3: Choose an **interval that overlaps with the fewest other intervals**, take it, remove overlaps, iterate on remainder.

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

Foundations of CS

Blah

Applied Topology with Doughnuts

Advanced Blah

Counterexample:

Professor Y's Greedy Algorithms



Attempt 3: Choose an **interval that overlaps with the fewest other intervals**, take it, remove overlaps, iterate on remainder.

Untangling String Theory

Poetry for Physicists

REM Lab

Aliens

Culture of Hieroglyphs

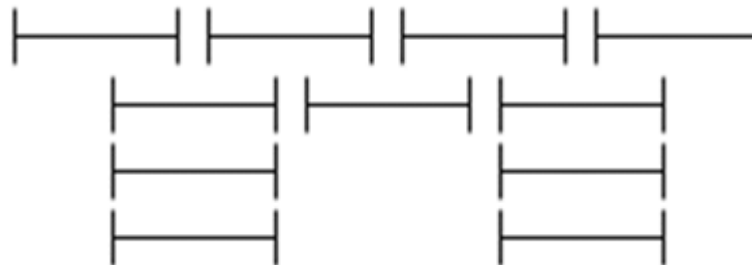
Foundations of CS

Blah

Applied Topology with Doughnuts

Advanced Blah

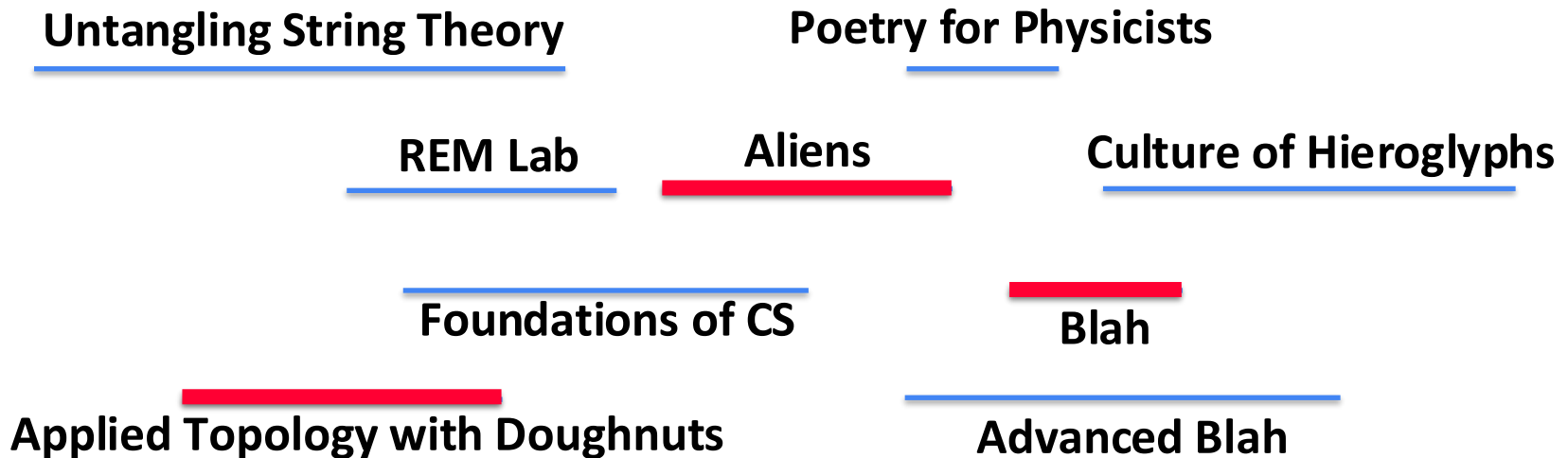
Counterexample:



A Correct Greedy Algorithm: “Earliest Ending Time (EET)”

- (Assume tasks are sorted by *ending time*)
- Greedily take an **interval I that ends first** (break ties arbitrarily)
- Remove the intervals that overlap with I
- Iterate on remaining intervals

Running time: $O(n)$



A Correct Greedy Algorithm: “Earliest Ending Time (EET)”

- (Assume tasks are sorted by *ending time*)
- Greedily take an **interval I that ends first** (break ties arbitrarily)
- Remove the intervals that overlap with I
- Iterate on remaining intervals

Running time: $O(n)$



A Correct Greedy Algorithm: “Earliest Ending Time (EET)”

Procedure *task selection*($s_1, \dots, s_n, f_1, \dots, f_n$: start & finish times for
n tasks, sorted by finish time.

$S := \emptyset$

for $j := 1$ to n

if *task j is compatible with S*

$S := S \cup \{j\}$

return S

This short piece of pseudocode is our algorithm! Everything after this is analysis.

We'll see the key idea of the proof of correctness first and then the general proof afterwards



Proof that first chosen interval is “safe”

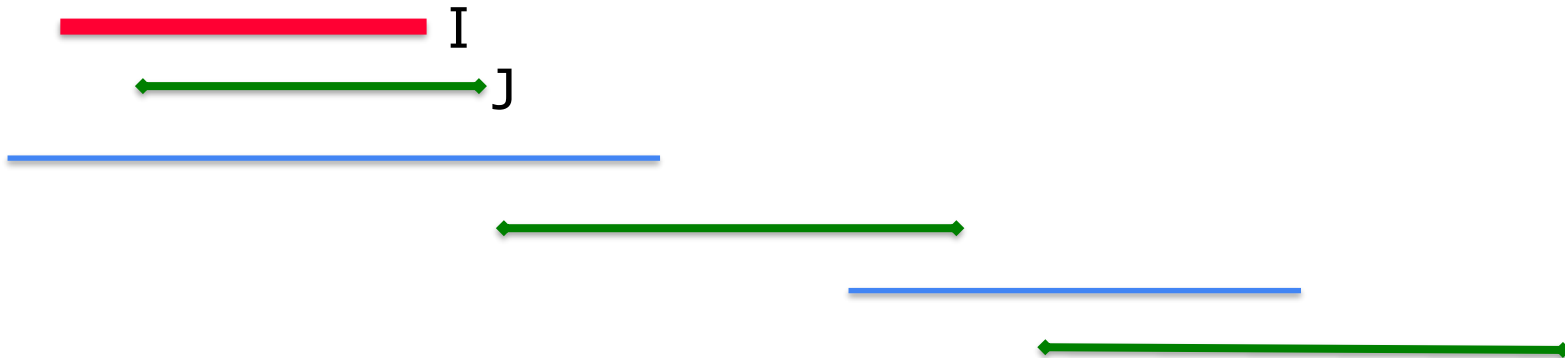
Key Claim: An interval **I** that **ends first** is a **safe** choice, i.e., it is in some optimal solution.

Proof: Consider any optimal solution **OPT**. Let **J** be an interval that ends earliest in **OPT**.

→ **J** ends no earlier than **I**: $\text{end}(\mathbf{I}) \leq \text{end}(\mathbf{J})$.

→ For every other interval **J'** in **OPT**: $\text{end}(\mathbf{J}) \leq \text{start}(\mathbf{J}')$,
so $\text{end}(\mathbf{I}) \leq \text{start}(\mathbf{J}')$.

→ Thus, we can **exchange J for I** in **OPT** and get a valid solution **OPT'** of the same (optimal) size!



Proof that **all** chosen intervals are “safe”

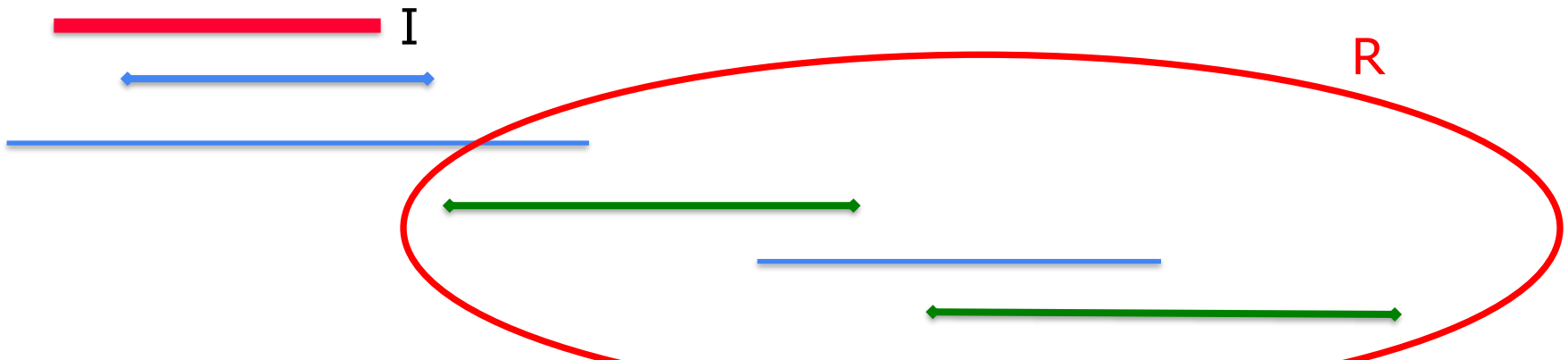
We know: An interval **I** that **ends first** is a **safe** choice, i.e., it is in some optimal solution **OPT**.

Now:

- Remove **I** and its overlaps. Call the remaining instance **R**.
- The rest of **OPT** must be optimal for **R**.

Otherwise, improve it and keep **I** to improve the original optimal.

- By induction, greedy solves the smaller instance **R** optimally too (and therefore, the original problem).



General strategy commonly used for analyzing greedy algorithms:

Exchange argument

The idea: Show that we can transform any **optimal solution** into the **solution given by our algorithm** by **exchanging** each piece of it out one-by-one so that:

- the value doesn't become worse
- the solution remains valid

All this exchanging is in the *analysis of correctness*—not part of the algorithm itself!

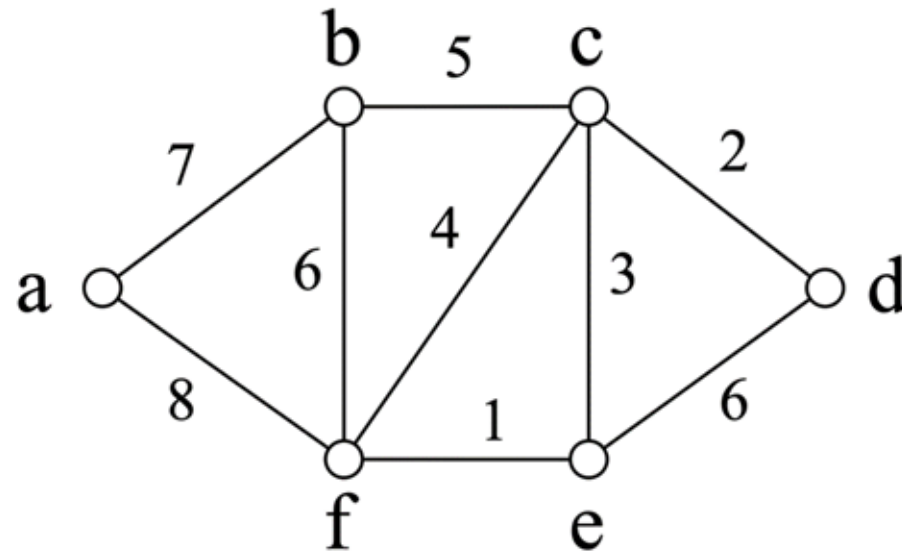


Another problem where greed is good...

A Highway Problem

Input: an undirected connected graph with positive edge weights, e.g., a set of cities and distances between them

Output: connect all cities using minimum total length of highway, i.e., it should be possible to reach any city from any other using the highways



a and **d** are Ann Arbor and Detroit, resp.

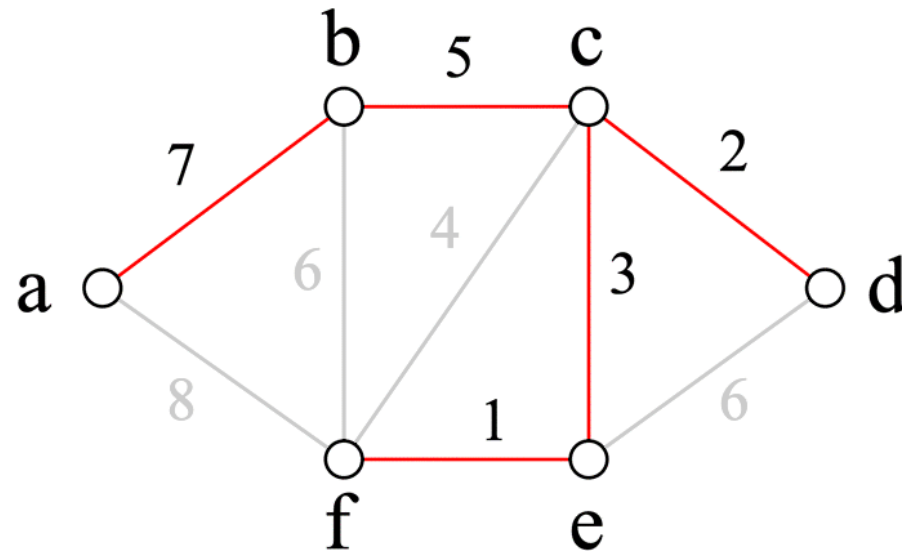


Another problem where greed is good...

A Highway Problem

Input: an undirected connected graph with positive edge weights, e.g., a set of cities and distances between them

Output: connect all cities using minimum total length of highway, i.e., it should be possible to reach any city from any other using the highways



a and d are Ann Arbor and Detroit, resp.



Another problem where greed is good...

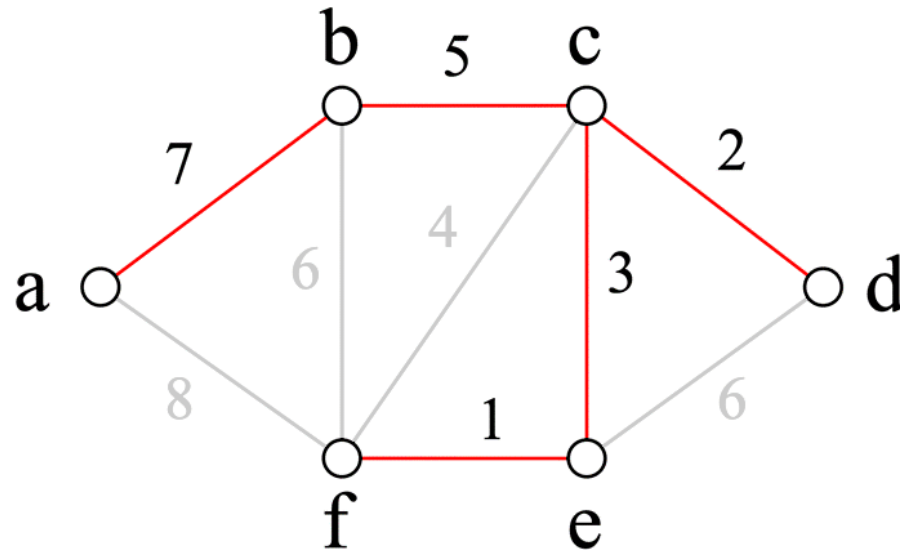
A Highway Problem

Claim: Solution is a tree.

↙ This fact will be useful later too

Why? If a connected graph has a cycle, we can delete any edge from the cycle (reducing the cost) and still have a connected graph.

Definition (review): A tree is a connected graph with no cycle.

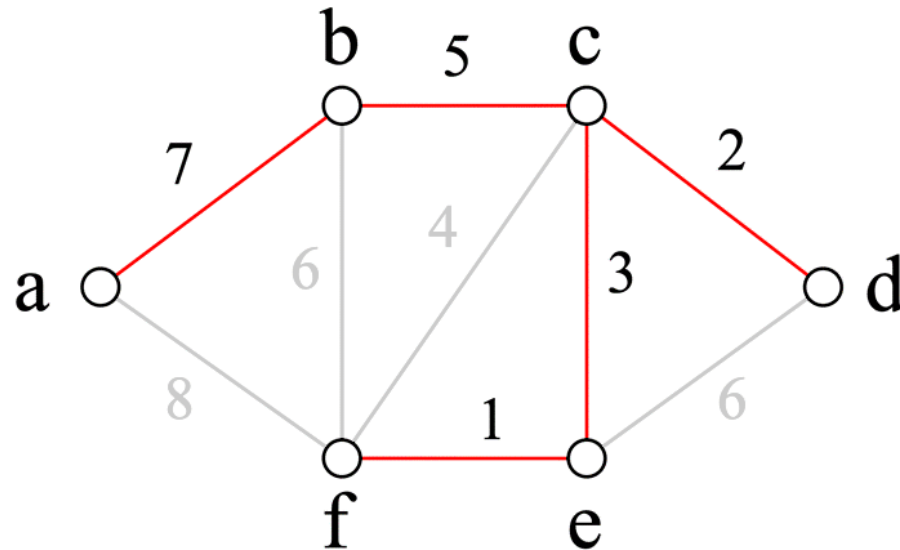


The Highway Problem is commonly known as:

Minimum Spanning Tree (MST)

Given a graph G , a **spanning tree** is a subgraph of G that is spanning (includes all vertices) and is a tree.

MST problem: Given an undirected connected graph with positive edge weights, find a minimum(-weight) spanning tree.

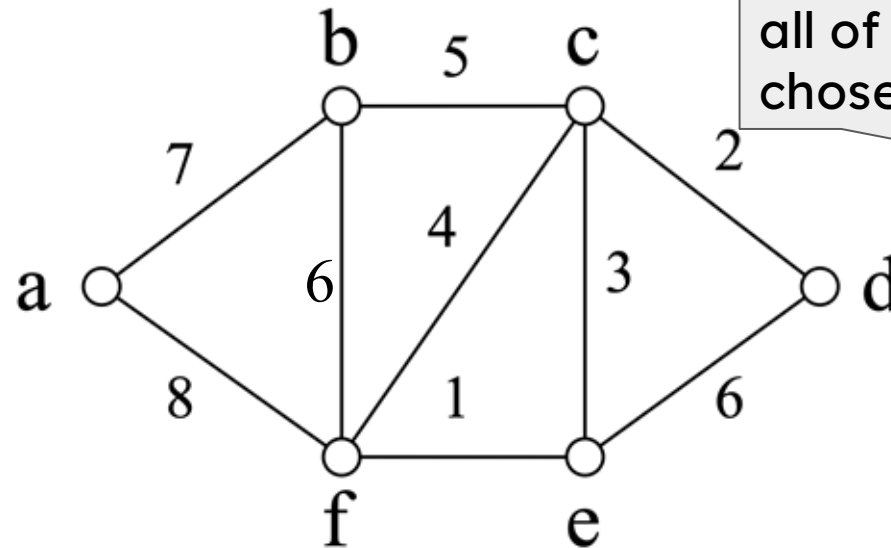


Minimum Spanning Tree (MST)

Template for greedy algorithm:

Greedly choose an edge and add it to our set. Repeat until we have a spanning tree.

But which edge to choose?



We need to choose a **safe** edge: an edge that appears in *some* optimal solution with all of the previously chosen edges.



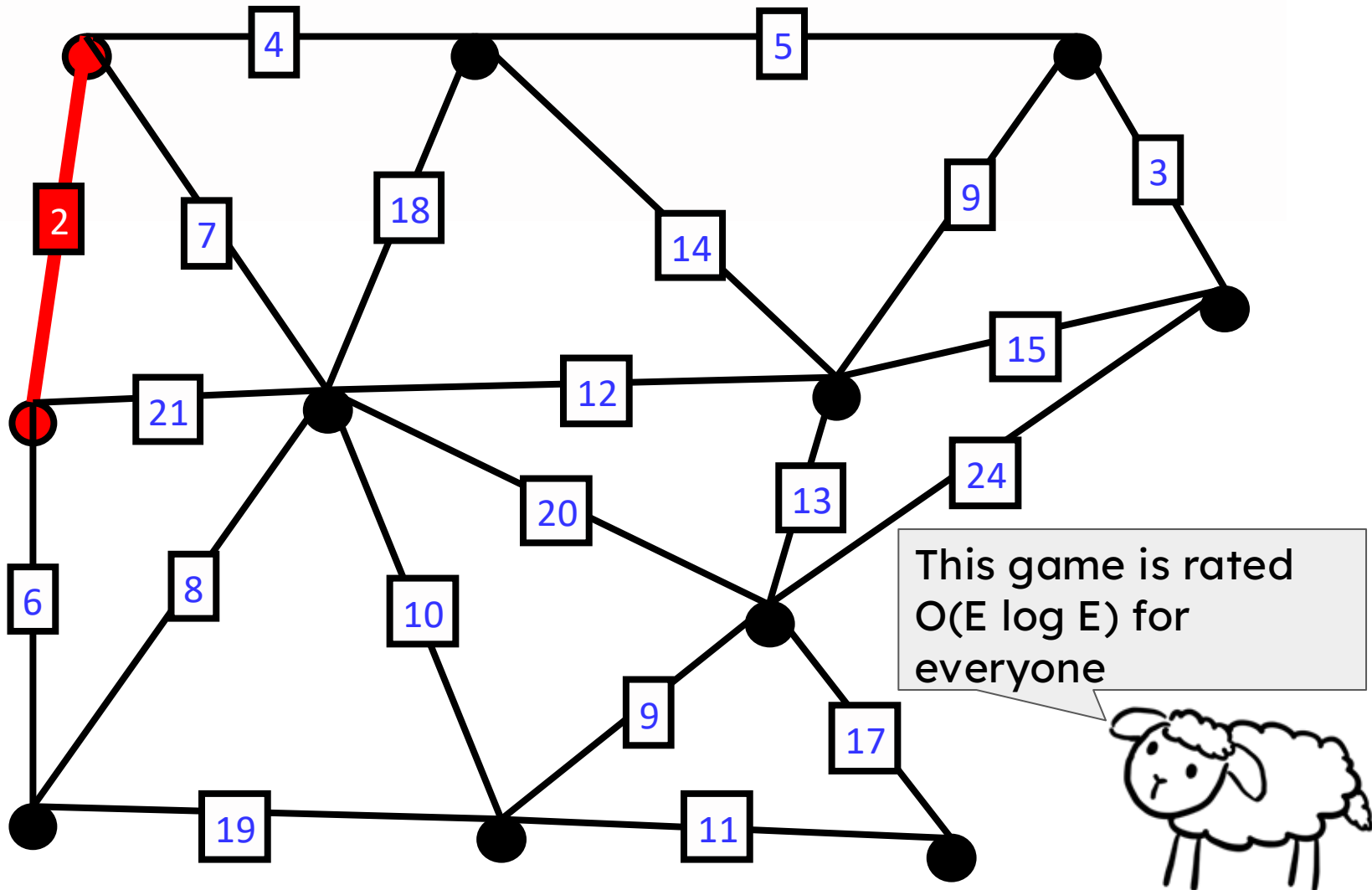
Kruskal's Algorithm (1956)

Choose an edge of minimum weight
(that doesn't introduce a cycle)!

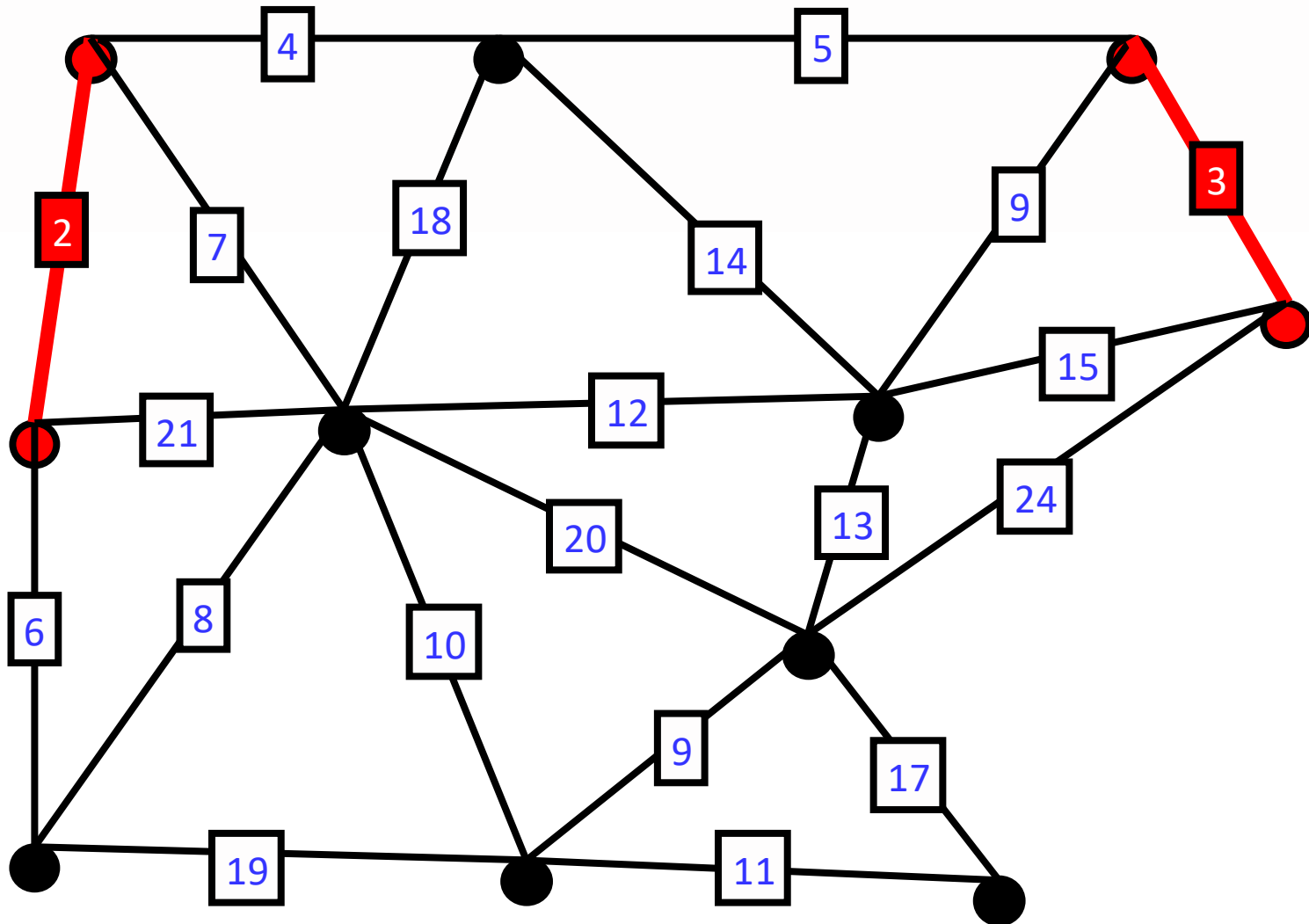


Kruskal(G): // G is a weighted, undirected graph
 $T \leftarrow V$ // T starts as just the vertices of G with no edges
// invariant: T has no cycles
for each edge e in *increasing order of weight*:
 if $T + e$ is acyclic: $T \leftarrow T + e$
return T

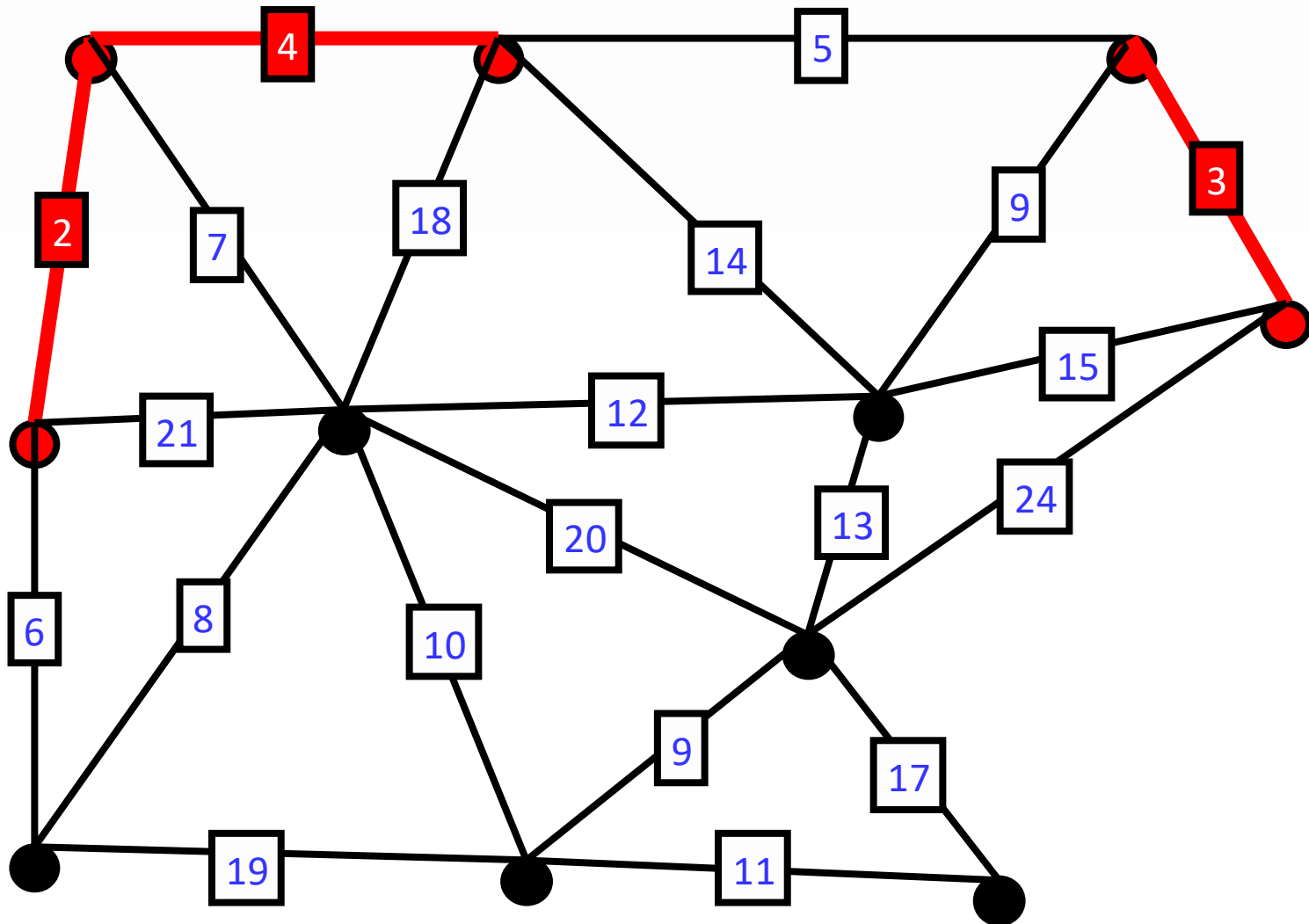
Sorted weights: **2**,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



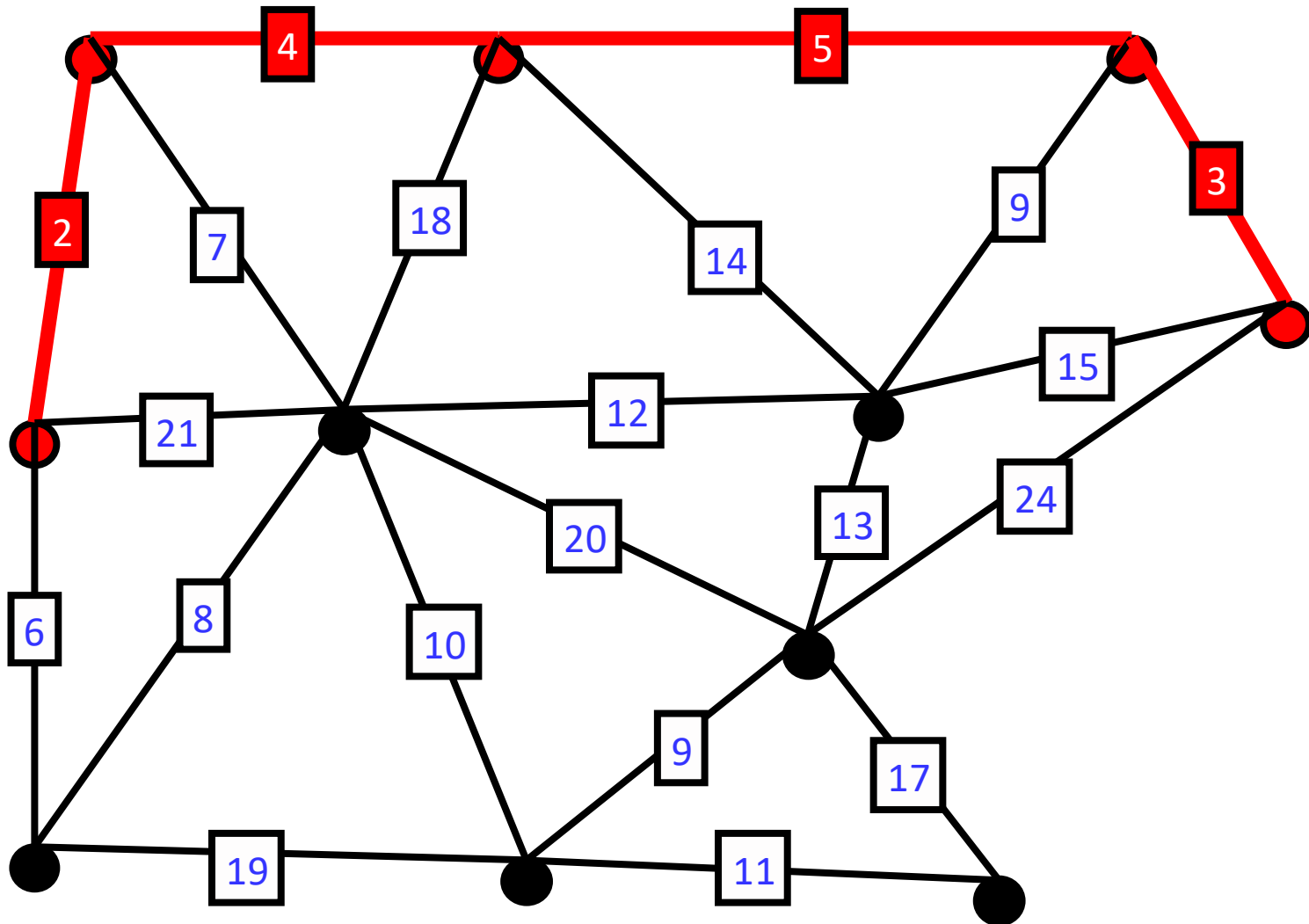
Sorted weights: 2, **3**, 4, 5, 6, 7, 8, 9, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 24



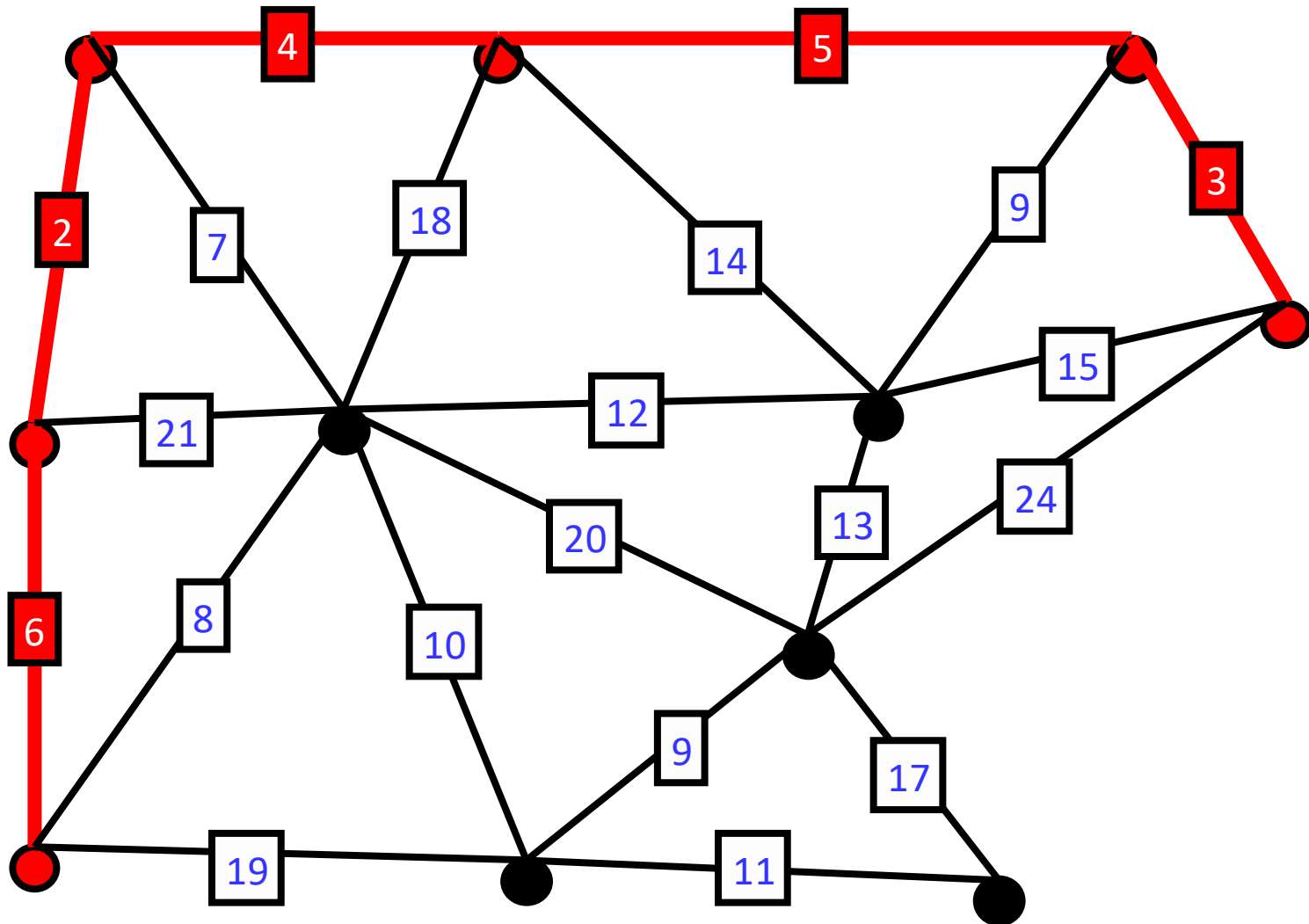
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



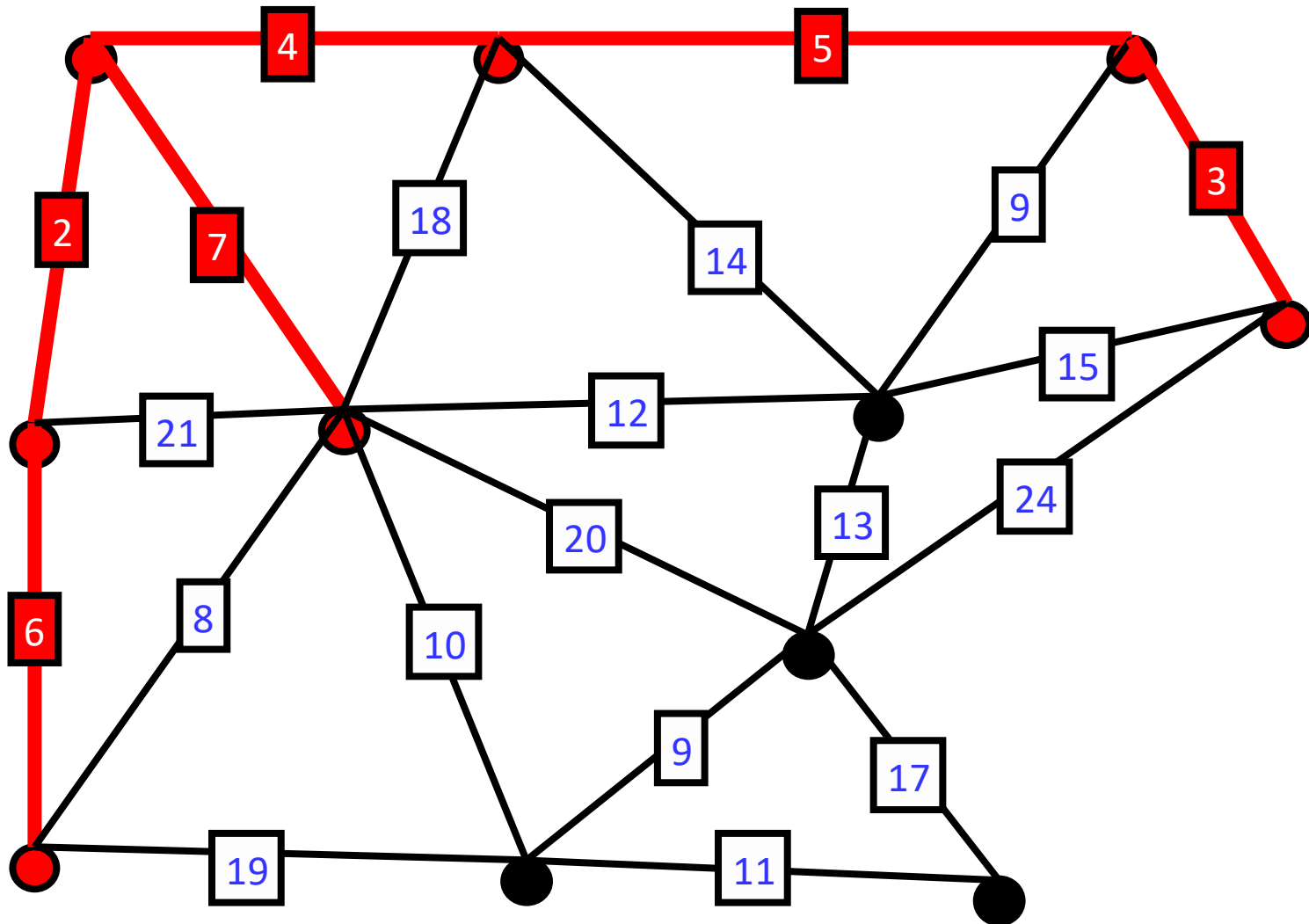
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



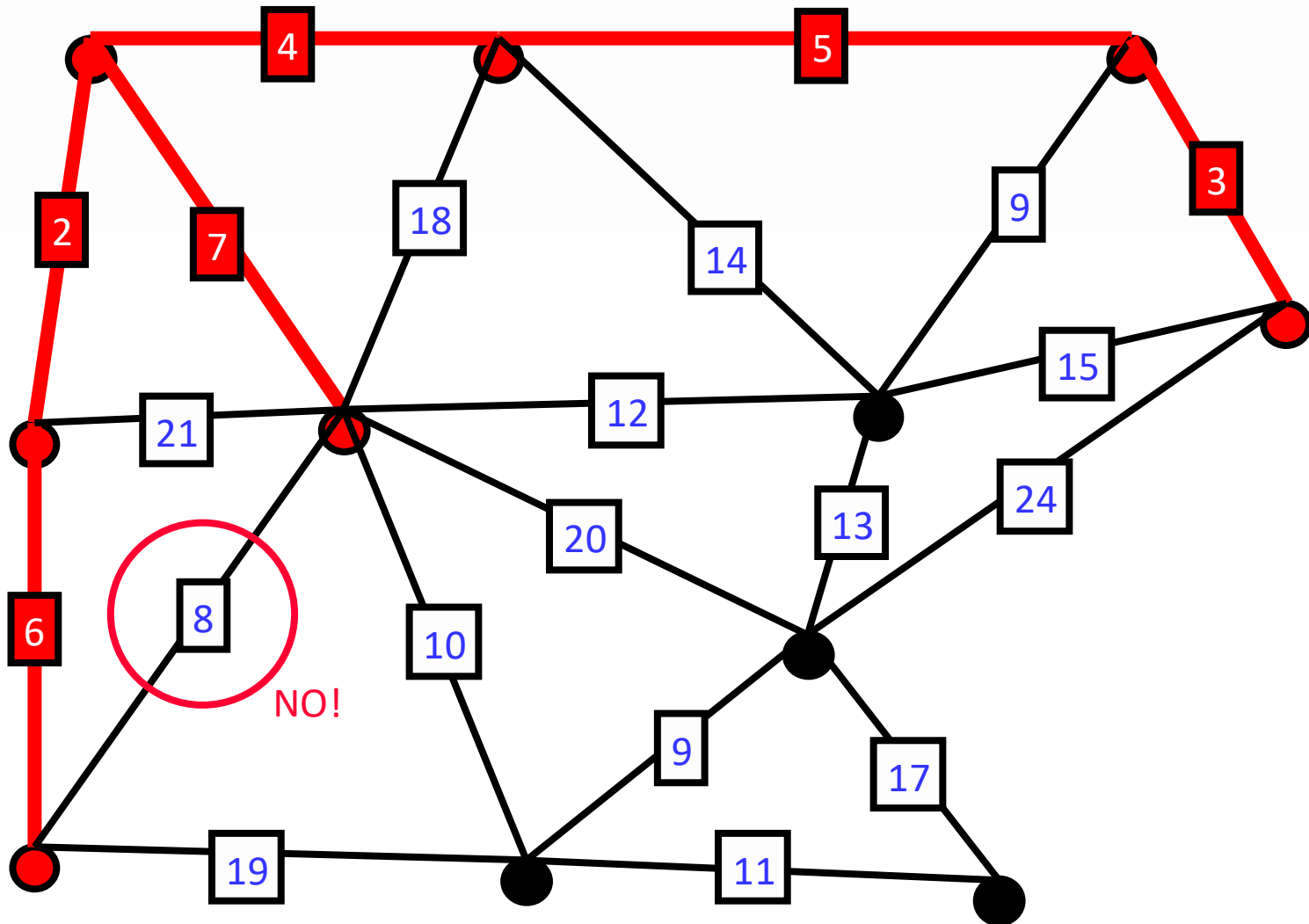
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



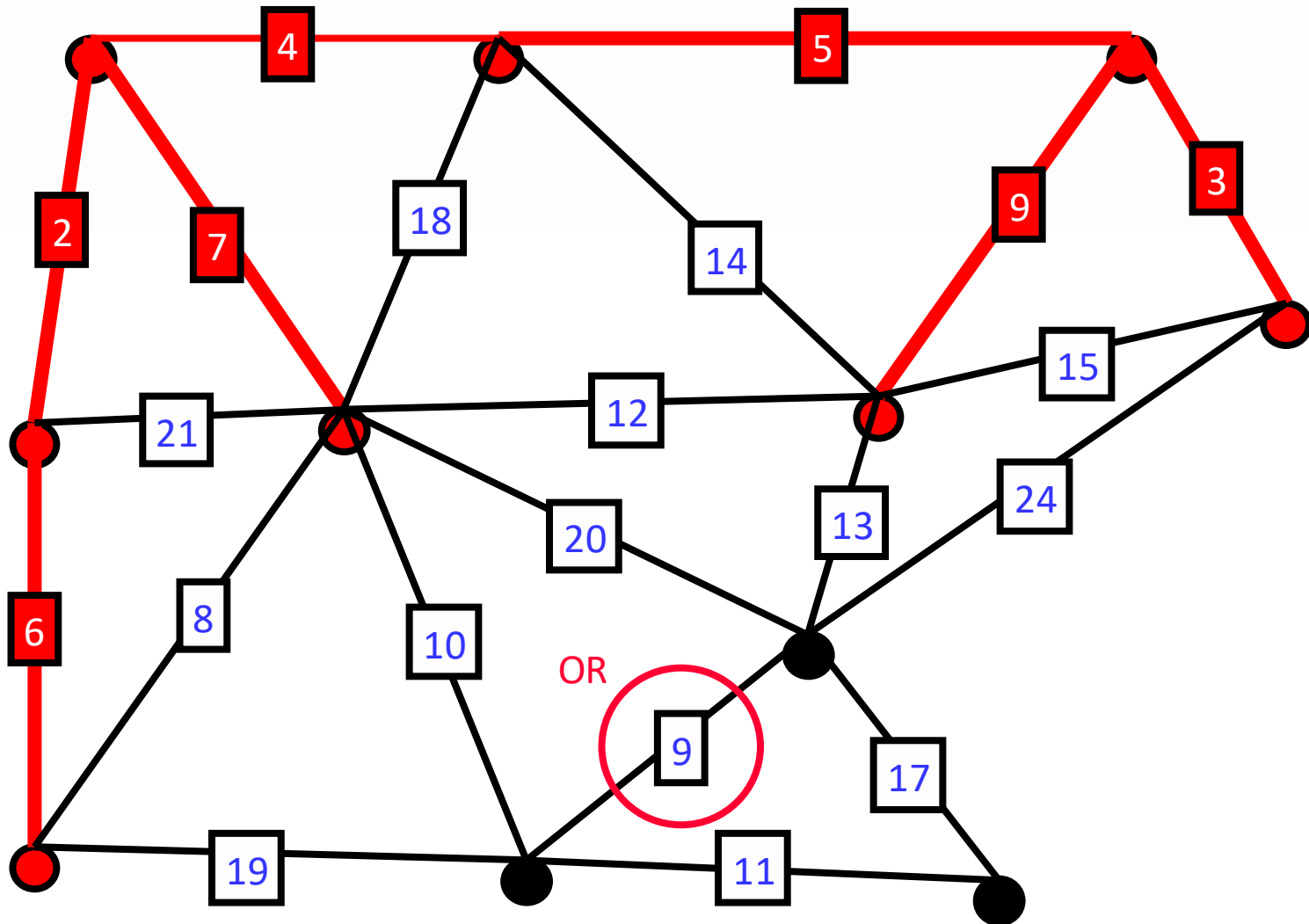
Sorted weights: 2,3,4,5,6,**7**,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



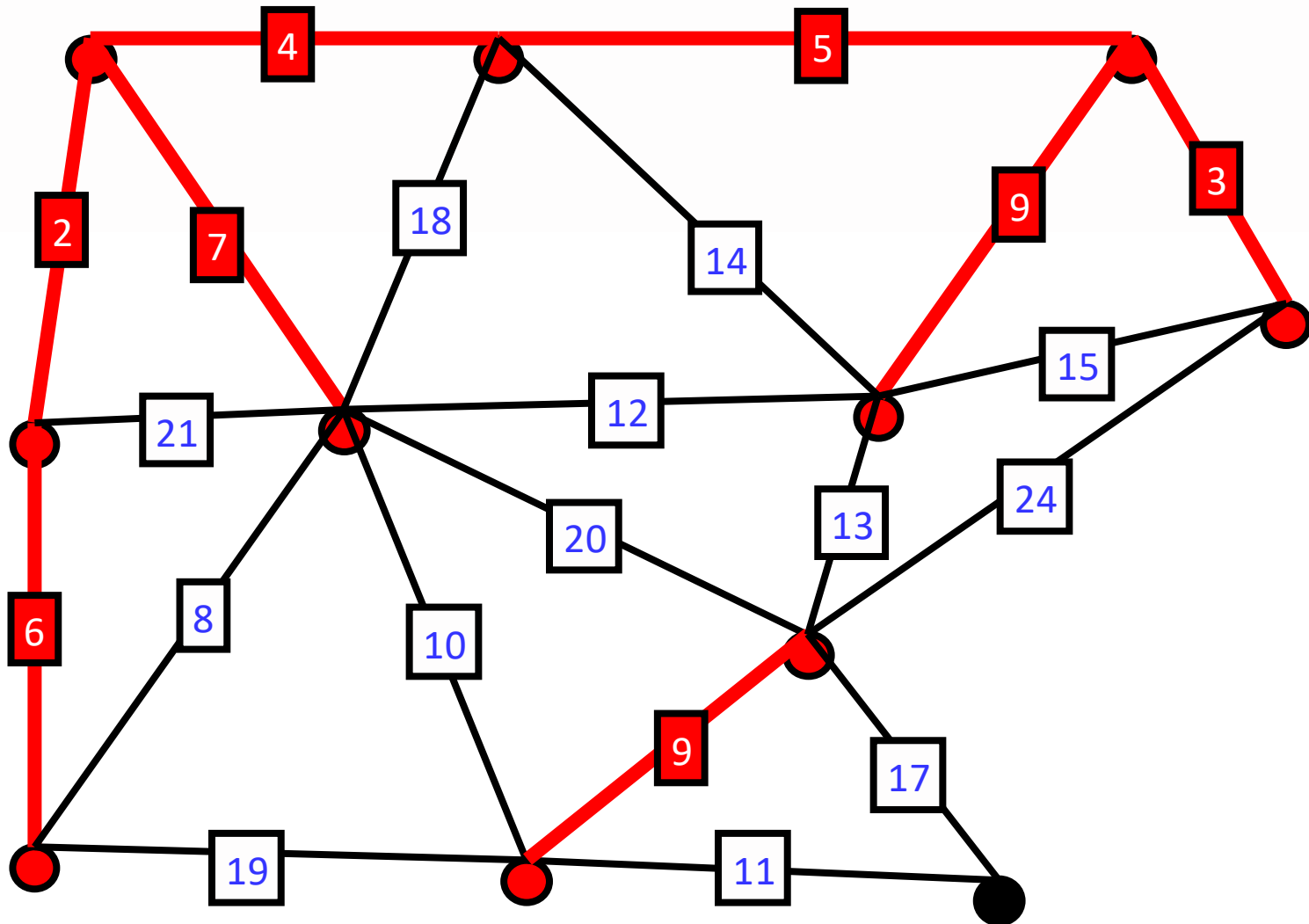
Sorted weights: 2,3,4,5,6,7,**8**,9,9,10,11,12,13,14,15,17,18,19,20,21,24



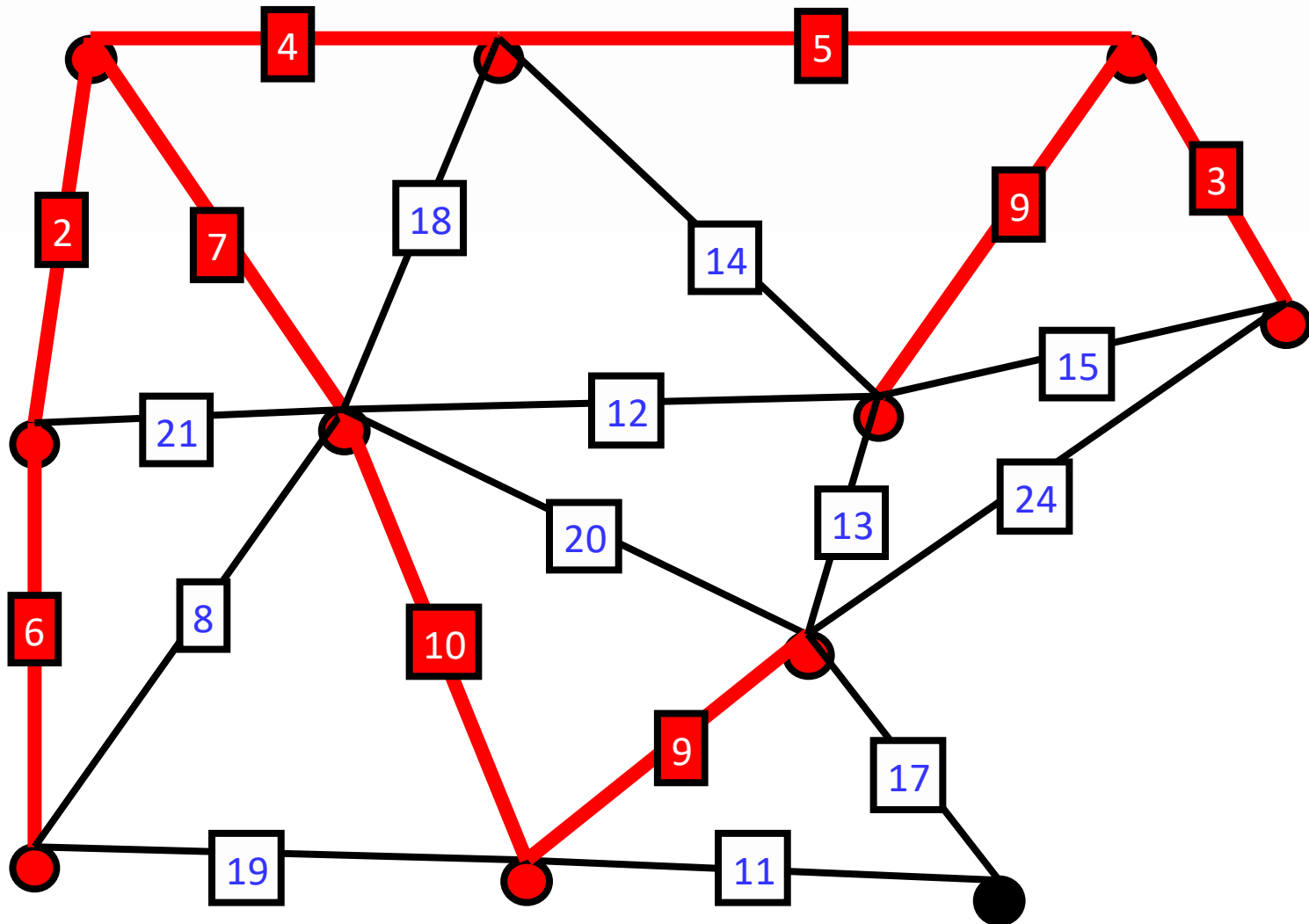
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



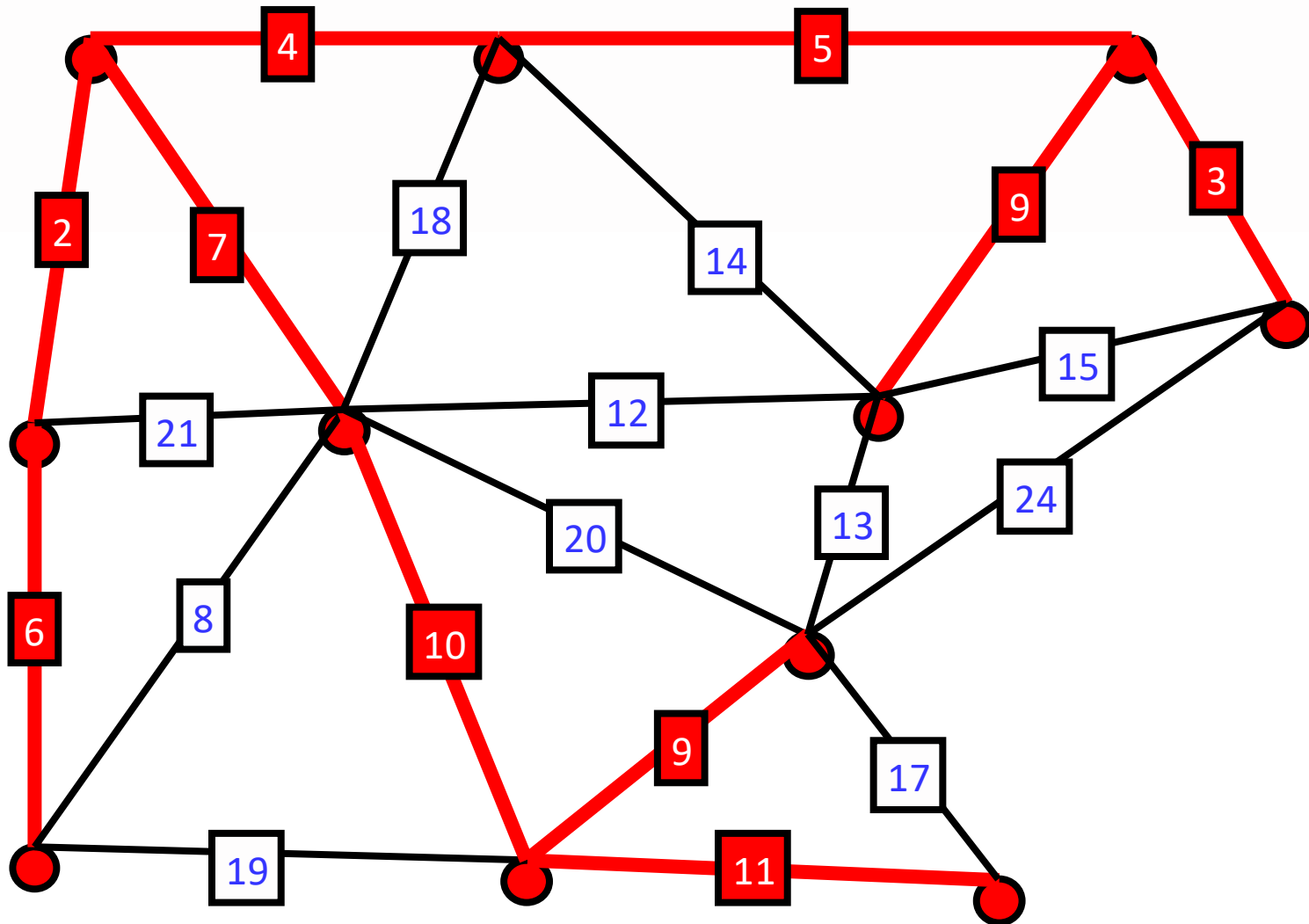
Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,12,13,14,15,17,18,19,20,21,24



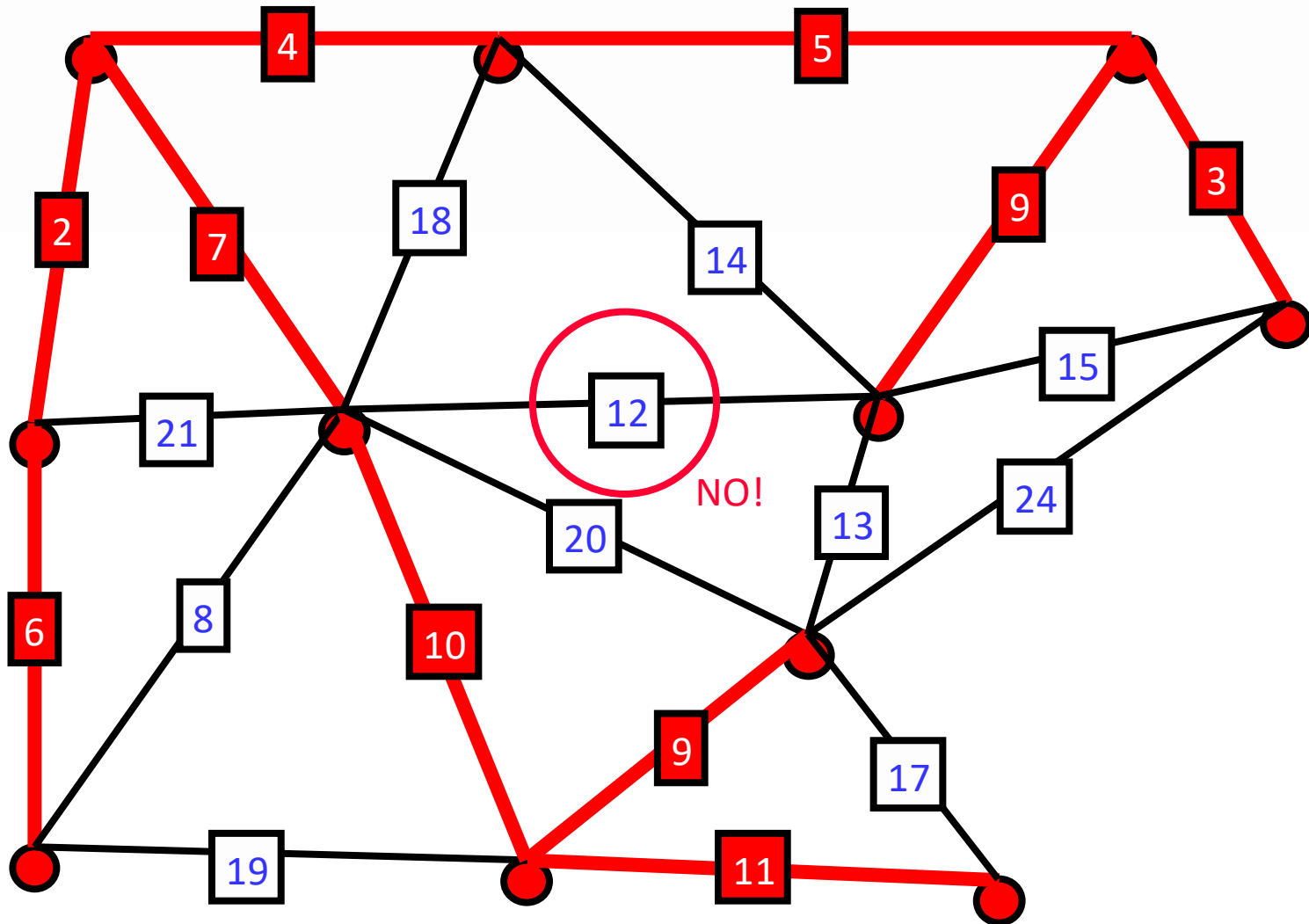
Sorted weights: 2,3,4,5,6,7,8,9,9,**10**,11,12,13,14,15,17,18,19,20,21,24



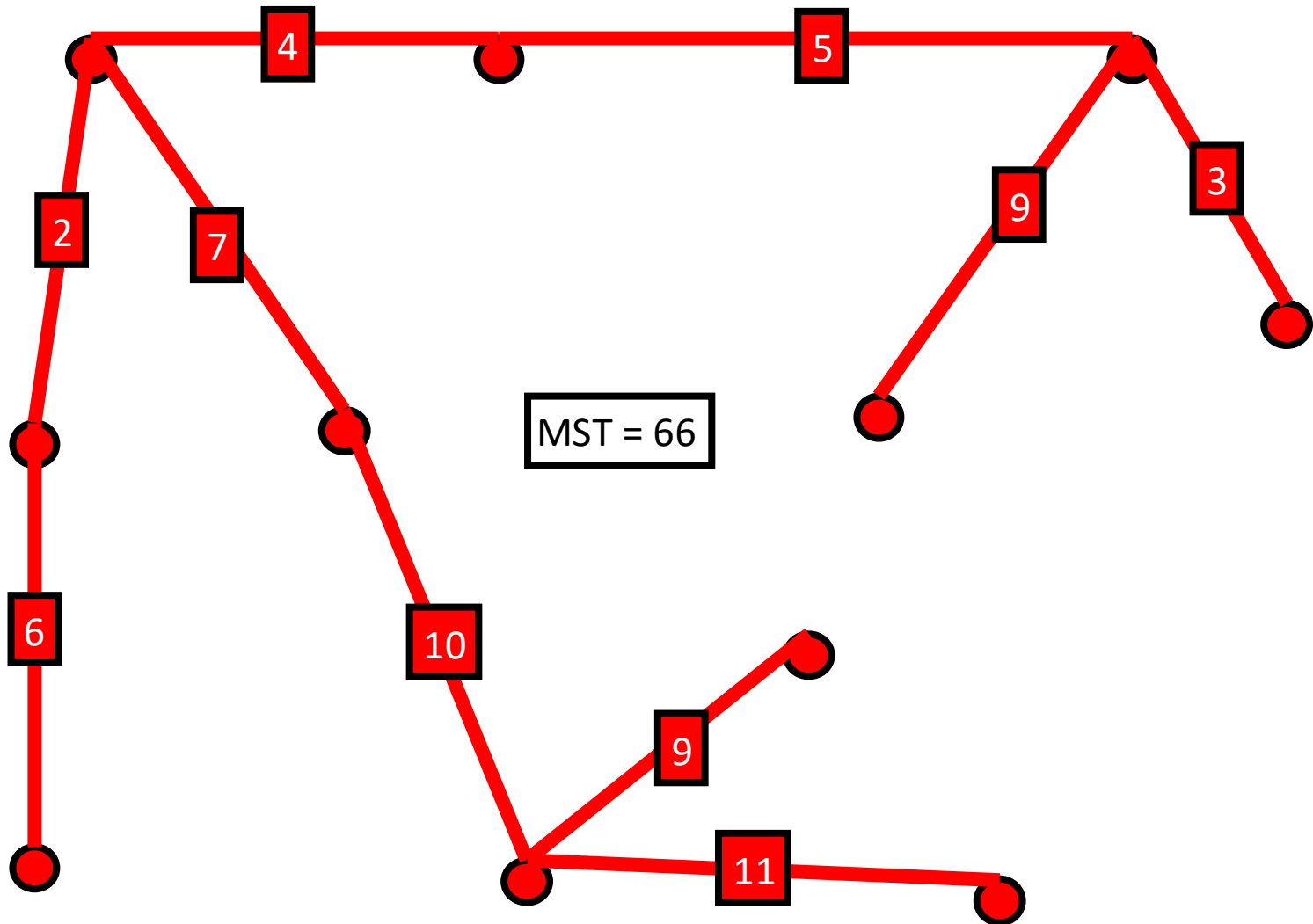
Sorted weights: 2,3,4,5,6,7,8,9,9,10,**11**,12,13,14,15,17,18,19,20,21,24



Sorted weights: 2,3,4,5,6,7,8,9,9,10,11,**12**,13,14,15,17,18,19,20,21,24



Kruskal's Algorithm



Kruskal's Algorithm: Correctness

Why does Kruskal's algorithm return a **spanning tree**?

1. Why is it **spanning** (i.e., connected / uses all vertices)?

(If not connected, there's an edge between two components since G is connected. That edge would have been considered and added at some point by the algorithm, a contradiction.)

2. Why is it a **tree** (i.e. connected with no cycles)?

(By construction.)

3. Why does Kruskal's algorithm return a **minimum** spanning tree?

Again, we'll use an **exchange** argument.

Kruskal's Algorithm: Correctness

Let **ALG** be the output of Kruskal's algorithm

Let **OPT** be a hypothetical optimal solution (if **ALG**=**OPT**, we are done)

Consider the **first edge** e_{diff} added by **Kruskal's algorithm** that's not in **OPT**.

$e_1, e_2, \dots, e_{\text{diff}}, \dots = \text{ALG}$ (ordered by weight)

$\underbrace{e_1, e_2, \dots}_{\text{same as ALG}}, \underbrace{f_{\text{diff}}, \dots}_{\text{some edges}} = \text{OPT}$

same as ALG some edges (may or may not be in **ALG**, in no particular order)

Key Claim: There's an optimal solution **OPT'** containing $e_1, e_2, \dots, e_{\text{diff}}$.
(will prove)

Consequence of Claim: We moved the “first difference” to later in **ALG**!
(I.e., first difference of **ALG** vs. **OPT'** is later than that of **ALG** vs. **OPT**)
Keep repeating $\Rightarrow$ **ALG** is an optimal solution!

Kruskal's Algorithm: Correctness

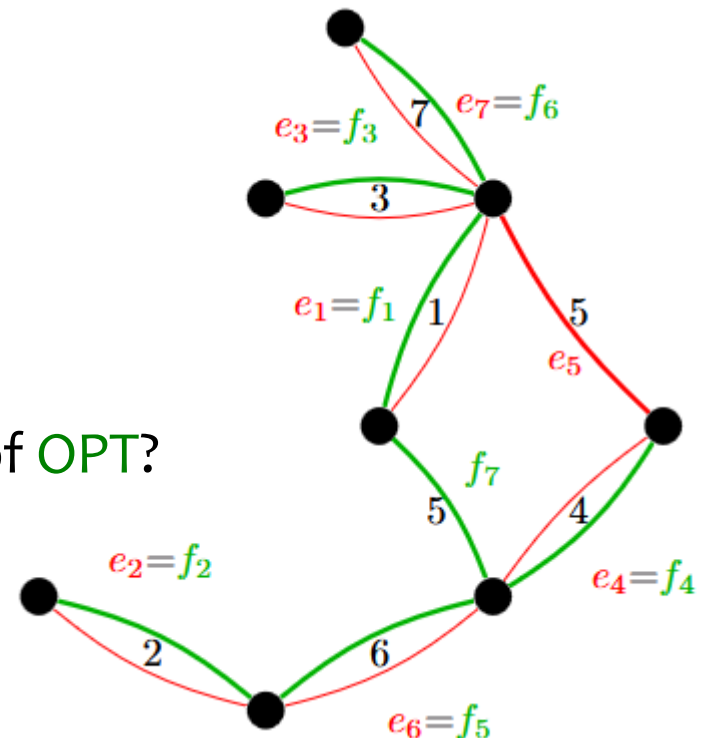
Key Claim: There's an optimal solution OPT' containing $e_1, e_2, \dots, e_{\text{diff}}$.

Proof of Claim:

Goal: Take OPT and **exchange** out **some edge f** for e_{diff} to form another optimal solution OPT' . That is:

- f in $\text{OPT} \setminus \{e_1, e_2, \dots, e_{\text{diff}-1}\}$
- OPT' should be a (spanning) tree
- $\text{weight}(\text{OPT}') \leq \text{weight}(\text{OPT})$
[actually must be = because OPT is optimal]

Which edge f should we **exchange** out of OPT ?



Kruskal's Algorithm: Correctness (continued):

$OPT \cup \{e_{diff}\}$ must have a cycle, call it C

ALG does not have a cycle, so there must be at least one edge in C that is not in ALG , call it f

Form OPT' by taking OPT and exchanging e_{diff} for f

After the exchange, why is the graph still a tree?

Still connected? Still no cycles?

Why did the exchange not increase the weight of the spanning tree?

Want to show: $\text{weight}(e_{diff}) \leq \text{weight}(f)$.

Our algorithm picks the smallest weight that doesn't cause a cycle. When we chose e_{diff} , we could have chosen f , so $\text{weight}(e_{diff}) \leq \text{weight}(f)$

Other independent inventors of MST algorithms:

Jarník, Prim, Borůvka



First used for designing electrical networks in 1926!

Today, a number of algorithms for clustering data (useful in machine learning) are variations of Kruskal's algorithm

Running time of Kruskal's algorithm: $O(m \log m)$ [we won't prove]

★ need a fast data structure for detecting cycles

Best-known running time (Chazelle 2000): $O(m \cdot \alpha(n))$

Inverse Ackermann: an
extreeeemely slow-
growing function:
 $\alpha(n) \leq 4$ even when $n =$
particles in known
universe

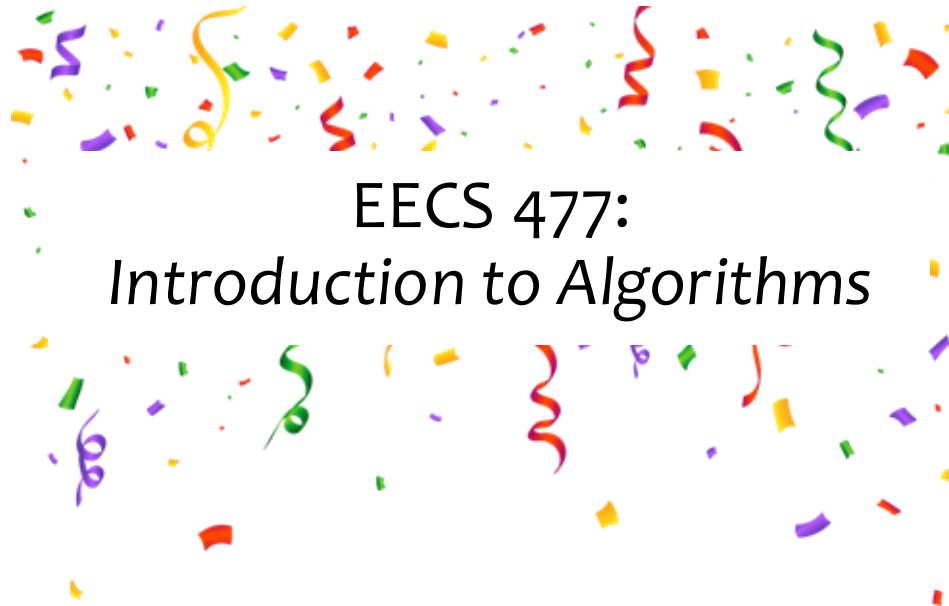
Open problem: $O(m)$?

Seth Pettie and Vijaya Ramachandran (2002) gave an asymptotically
fastest possible algorithm. But nobody knows how fast it is!



You've just finished a crash course on algorithms!

If you found the material interesting, try:



Next up: **Computability**

- What is a computer?
- What problems can a computer solve?
- What problems can a computer **not** solve?